Garcia Sad Tomás Agustín
Legajo: 14077

Trabajo práctico N°4

Ejercicio 1

Parte 1

```
Controlador ctrl;
FiguraOriginal figura;

void setup() {
  size(800, 600, P3D); // Declaración explícita del motor 3D
  ctrl = new Controlador();
  figura = new FiguraOriginal();
}

void draw() {
  background(30);

  // Configuración de iluminación direccional y ambiental para realzar el contraste tridimensional
  lights();
  directionalLight(200, 200, 255, 1, 1, -1);
  ambientLight(50, 50, 50);

  // Traslación al centro del entorno gráfico
  translate(width / 2, height / 2, 0);

  // Actualización del estado lógico y aplicación de rotaciones
  ctrl.actualizarRotacion();
  rotateX(ctrl.rotX);
  rotateY(ctrl.rotY);

  // Despliegue de la geometría con el patrón activo
  figura.mostrar(ctrl.patronActual);
}

void keyPressed() {
  ctrl.procesarTecla(keyCode, key);
}


// --- ARQUITECTURA DE CLASES ---

class Controlador {
  // Se inicializa con un ángulo isométrico (PI/4) para evidenciar la profundidad 3D desde el primer frame
  float rotX = QUARTER_PI;
  float rotY = QUARTER_PI;
  float velX = 0, velY = 0;
  final float VELOCIDAD = 0.03;
  int patronActual = 0;

  void actualizarRotacion() {
    rotX += velX;
    rotY += velY;
  }

  void procesarTecla(int cod, char t) {
    // Control de sentido de rotación mediante flechas direccionales
    if (cod == UP) {
      velX = -VELOCIDAD; velY = 0;
    } else if (cod == DOWN) {
      velX = VELOCIDAD; velY = 0;
    } else if (cod == LEFT) {
      velY = -VELOCIDAD; velX = 0;
    } else if (cod == RIGHT) {
      velY = VELOCIDAD; velX = 0;
    }

```

```
    char tl = Character.toLowerCase(t);
    if (tl == 'd') {
      // Detener rotación instantáneamente
      velX = 0;
      velY = 0;
    } else if (tl == 'r') {
      // Reinicio del sistema: ángulos a posición isométrica inicial, detención de movimiento y cambio de textura
      rotX = QUARTER_PI;
      rotY = QUARTER_PI;
      velX = 0;
      velY = 0;
      patronActual = (patronActual + 1) % 3;
    }
  }
}

class FiguraOriginal {
  PImage pat1, pat2, pat3;
  float tamanio = 180;

  FiguraOriginal() {
    // Construcción algorítmica de patrones
    pat1 = crearPatronAjedrez();
    pat2 = crearPatronRayas();
    pat3 = crearPatronRuido();
  }

  void mostrar(int idPatron) {
    PImage tex = pat1;
    if (idPatron == 1) tex = pat2;
    if (idPatron == 2) tex = pat3;

```

```
    textureMode(NORMAL);
    noStroke();

    // Renderizado del Plano 1 (Corte XY)
    pushMatrix();
    dibujarPlano(tex);
    popMatrix();

    // Renderizado del Plano 2 (Corte XZ)
    pushMatrix();
    rotateX(HALF_PI);
    dibujarPlano(tex);
    popMatrix();

    // Renderizado del Plano 3 (Corte YZ)
    pushMatrix();
    rotateY(HALF_PI);
    dibujarPlano(tex);
    popMatrix();



    // --- Agregar la figura central --
    pushMatrix();
    noFill();
    stroke(255, 255, 255, 150);
    strokeWeight(1);
    //sphereDetail(24);
    //sphere(120);
    box(170);
    popMatrix();
```

```
  }

  void dibujarPlano(PImage t) {
    beginShape(QUADS);
    texture(t);
    // Asignación de vértices espaciales y coordenadas de textura (Mapeo UV)
    vertex(-tamanio, -tamanio, 0, 0, 0);
    vertex( tamanio, -tamanio, 0, 1, 0);
    vertex( tamanio,  tamanio, 0, 1, 1);
    vertex(-tamanio,  tamanio, 0, 0, 1);
    endShape();
  }

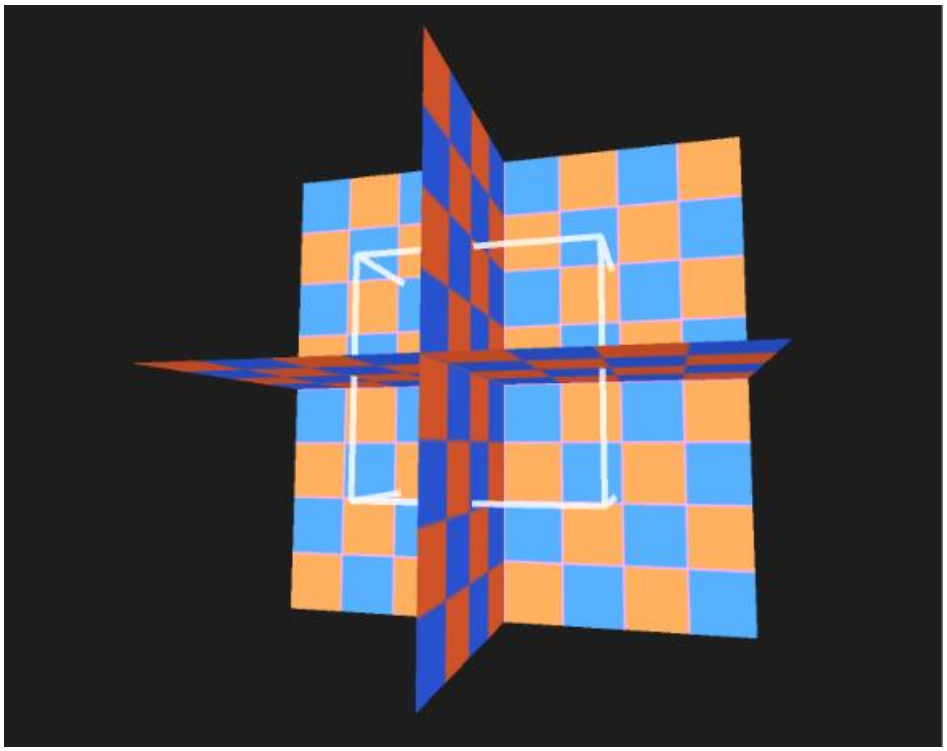
  PImage crearPatronAjedrez() {
    PImage img = createImage(128, 128, RGB);
    img.loadPixels();
    for (int i = 0; i < img.width; i++) {
      for (int j = 0; j < img.height; j++) {
        boolean cuadroPar = ((i / 16) % 2 == (j / 16) % 2);
        img.pixels[j * img.width + i] = cuadroPar ? color(255, 100, 50) : color(50, 100, 255);
      }
    }
    img.updatePixels();
    return img;
  }

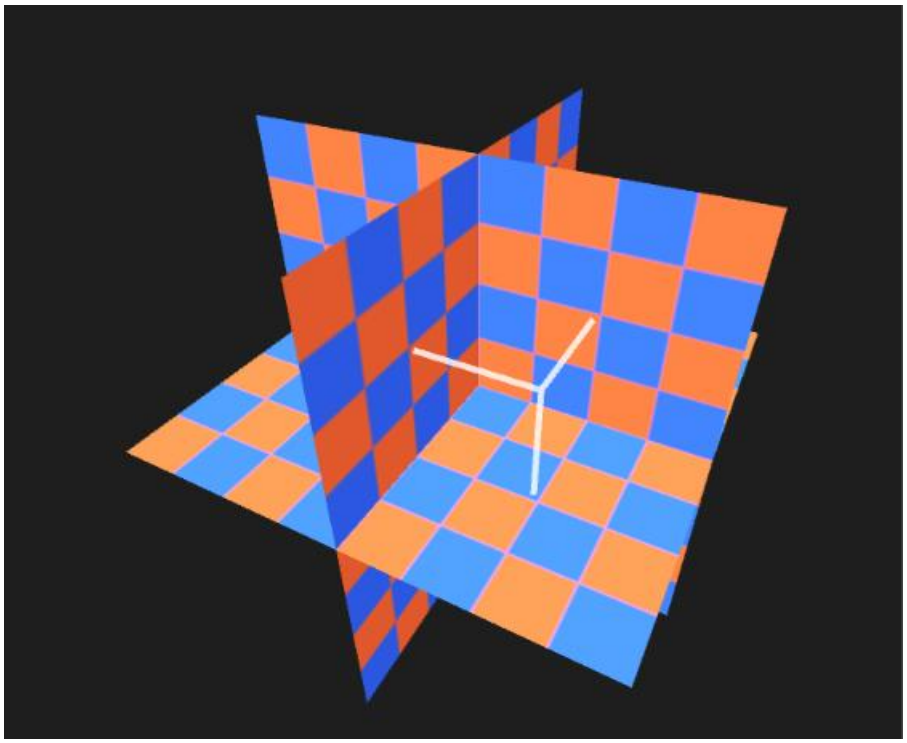
  PImage crearPatronRayas() {
    PImage img = createImage(128, 128, RGB);
    img.loadPixels();
    for (int i = 0; i < img.width; i++) {

      for (int j = 0; j < img.height; j++) {
        img.pixels[j * img.width + i] = ((i + j) % 32 < 16) ? color(50, 255, 100) : color(20);
      }
    }
    img.updatePixels();
    return img;
  }

  PImage crearPatronRuido() {
    PImage img = createImage(128, 128, RGB);
    img.loadPixels();
    for (int i = 0; i < img.pixels.length; i++) {
      img.pixels[i] = color(random(255), random(255), random(255));
    }
    img.updatePixels();
    return img;
  }
}
```

Garcia Sad Tomás Agustín
Legajo: 14077





Garcia Sad Tomás Agustín
Legajo: 14077

